

Anagram Detector Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-anagram-detector-lab>  <https://github.com/learn-co-curriculum/python-p3-anagram-detector-lab/issues/new>

Learning Goals

- Build robust and dynamic Python objects.
 - Accomplish complex programming tasks using knowledge from previous modules.
-

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
 - **Object**: the more common name for an instance. The two can usually be used interchangeably.
 - **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
 - **Function**: a series of steps that create, transform, and move data.
 - **Method**: a function that is defined inside of a class.
-

Instructions

This is a **test-driven lab**. Run `pipenv install` to create your virtual environment and `pipenv shell` to enter the virtual environment. Then run `pytest -x` to run your tests. Use these instructions and `pytest` 's error messages to complete your work in the `lib/` folder.

You will write a program that, given a word and a list of possible [anagrams](http://www.dictionary.com/browse/anagram) , selects the correct one(s).

Your class, `Anagram` should take a word on initialization; instances should respond to a `match()` instance method that takes a list of possible anagrams. It should return all matches in a list. If no matches exist, it should return an empty list.

In other words, given: `'listen'` and `['enlists', 'google', 'inlets', 'banana']` the program should return `['inlets']`.

```
listen = Anagram("listen")
listen.match(['enlists', 'google', 'inlets', 'banana'])
# => ['inlets']
```

Write your solution in `lib/anagram.py`.

Once all of your tests are passing, commit and push your work using `git` to submit.

This is a difficult lab that will require some algorithmic thinking! Try writing some pseudocode and break the problem down into smaller steps before writing out your implementation.

Hints:

How will you determine if one word is an anagram for another?

You'll need to iterate over the list of words that the `match()` method takes as an argument. You will compare each word of that list to the word that the `Anagram` class is initialized with.

To determine one word is an anagram of another word, try determining if they are composed of the same letters. You can use a list interpretation on a string to get a list of its individual letters:

```
[letter for letter in "hello"]
# ["h", "e", "l", "l", "o"]
```

You can compare two lists using the `==`. For example:

```
[1, 2, 3] == [1, 2, 3]  
# => True
```

```
[1, 3, 2] == [1, 2, 3]  
# => False
```

Two lists are equal if they contain the same elements, in the same order. Remember that you can use `sorted()` on a list. This will help in your comparison:

```
sorted([1, 3, 2]) == sorted([3, 2, 1])  
# => True
```

Resources

- [5. Data Structures - Python](https://docs.python.org/3/tutorial/datastructures.html)  [\(https://docs.python.org/3/tutorial/datastructures.html\)](https://docs.python.org/3/tutorial/datastructures.html)
- [Python sorted\(\) Function - W3schools](#) 

This tool needs to be loaded in a new browser window

Load Anagram Detector Lab (CodeGrade) in a new window